

ICCT COLLEGES FOUNDATION INC

College of Engineering and Digital Technology

PROJECT NEXUS

[TITLE]

A Capstone Project Presented to the Faculty of the
College of Engineering and Digital Technology,
in Partial Fulfillment of the Requirements for the Course
Software Design

Presented by:

[Member Name] — [Role, e.g., Lead Developer]

[Member Name] — [Role]

[Member Name] — [Role]

A.Y. 2026-2027

Approval Sheet

This project, titled "[System Title]", prepared and submitted by [Team Members], in partial fulfillment of the requirements for the course Software Design, has been examined and is recommended for acceptance and approval.

Adviser

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Engineering

JONATHAN C. NAVARRO, MIT, PhD
PANEL

DENVER PANTE MIT, PhD
PANEL

Acknowledgment

[Optional. A brief note from the team thanking advisers, panel members, and contributors. One page maximum.]

Abstract

A concise (250–300 word) summary covering: the problem addressed, the proposed solution, the methodology/tech stack used, key results from testing, and the deployment outcome. The abstract is written LAST, after Chapter 4 results are final, even though it appears first in the paper.

Suggested keywords line beneath the abstract, e.g.: Keywords: campus system, Agile, MVC, [your tech stack].

Table of Contents

Approval Sheet.....2
Acknowledgment.....3
Abstract.....4
Table of Contents.....5
List of Tables.....7
List of Figures.....8
CHAPTER 1 – THE PROBLEM AND ITS BACKGROUND.....9
1.1 Introduction.....9
1.2 Statement of the Problem.....9
1.3 Objectives of the Study.....9
1.3.1 General Objective.....9
1.3.2 Specific Objectives.....9
1.4 Scope and Limitations.....9
1.5 Significance of the Study.....9
1.6 Definition of Terms.....9
CHAPTER 2 REVIEW OF RELATED LITERATURE.....10
2.1 Related Literature.....10
2.2 Related Systems / Studies.....10
2.3 Comparative Matrix.....10
2.4 Synthesis.....10
2.5 Feasibility Study.....10
2.5.1 Technical Feasibility.....10
2.5.2 Schedule Feasibility.....10
2.5.3 Operational Feasibility.....10
2.5.4 Economic Feasibility.....10
CHAPTER 3 REQUIREMENTS AND SYSTEM DESIGN.....11
3.1 Software Development Methodology.....11
3.2 Requirements Gathering.....11
3.2.1 User Stories.....11
3.2.2 Functional Requirements.....11
3.2.3 Non-Functional Requirements.....11
3.3 System Architecture Design.....11
3.4 UML Diagrams.....11
3.4.1 Use Case Diagram.....11
3.4.2 Activity Diagrams.....11
3.4.3 Class Diagram.....11

3.4.4 Entity-Relationship Diagram (ERD).....11
3.5 UI/UX Design.....12
3.5.1 High-Fidelity Prototype.....12
3.5.2 Design Rationale.....12
3.6 Alpha Build.....12
CHAPTER 4 SYSTEM IMPLEMENTATION AND TESTING.....13
4.1 Development Environment and Tech Stack.....13
4.2 Implementation Summary.....13
4.3 Software Testing.....13
4.3.1 Unit Testing.....13
4.3.2 Integration Testing.....13
4.3.3 User Acceptance Testing (UAT).....13
4.3.4 Bug Log.....13
4.4 Deployment and Publishing.....13
4.5 Results and Discussion.....13
CHAPTER 5 SUMMARY.....14
5.1 Summary of Findings.....14
5.2 Conclusion.....14
5.3 Recommendations.....14
References.....14
Appendices.....14
Appendix A — Source Code Repository.....14
Appendix B — User Manual.....14
Appendix C — UAT Forms and Raw Results.....14
Appendix D — Consultation / Adviser's Log.....14
Appendix E — Curriculum Vitae.....14

List of Tables

List every UML diagram, screenshot, ERD, and table by figure/table number, caption, and page number. Update this list last, once page numbers are final.

List of Figures

CHAPTER 1 – THE PROBLEM AND ITS BACKGROUND

1.1 Introduction

Describe the campus or community context, the current process or pain point, and why it matters. Ground this in observation, data, or interviews — not assumption.

1.2 Statement of the Problem

State the general problem in one or two sentences, then break it into specific sub-problems (usually 3–5), each addressable by a system feature.

1.3 Objectives of the Study

1.3.1 General Objective

One sentence describing the overall system to be built and its purpose.

1.3.2 Specific Objectives

List objectives using SMART criteria (Specific, Measurable, Achievable, Relevant, Time-bound), each tied directly to a sub-problem in 1.2.

1.4 Scope and Limitations

Define exactly what the system will and will not do, which user roles are supported, and any constraints (time, data access, hardware, budget).

1.5 Significance of the Study

Identify who benefits (students, faculty, admin, the institution) and how — direct beneficiaries first, then broader/indirect ones.

1.6 Definition of Terms

Define technical and domain-specific terms used throughout the paper, operationally (as used in this study), listed alphabetically.

CHAPTER 2 REVIEW OF RELATED LITERATURE

2.1 Related Literature

Summarize academic and industry sources relevant to your problem domain (e.g., prior research on campus systems, queuing, resource scheduling). Cite properly (APA/IEEE as required by your program).

2.2 Related Systems / Studies

Review existing applications or systems that address similar problems — commercial, open-source, or academic. For each: name, description, and a short comparison against your proposed system.

2.3 Comparative Matrix

Present a table comparing your proposed system against the related systems from 2.2 across key features. This directly supports your feasibility argument at Pitch Defense.

2.4 Synthesis

Explain the gap your project fills and how it differs from or improves upon existing solutions.

2.5 Feasibility Study

2.5.1 Technical Feasibility

Justify that the team has the skills and that the proposed tech stack (frontend, backend, database) can realistically deliver the required features.

2.5.2 Schedule Feasibility

Present a sprint-based timeline showing the project is achievable within the semester. A Gantt chart or sprint table is expected here.

2.5.3 Operational Feasibility

Assess whether end users (students, faculty, admin) will realistically adopt and be able to use the system.

2.5.4 Economic Feasibility

Estimate development and hosting costs (even if mostly free-tier) versus the value/savings the system provides.

CHAPTER 3 REQUIREMENTS AND SYSTEM DESIGN

3.1 Software Development Methodology

Describe the Agile approach adopted: sprint length, number of sprints planned for the semester, roles (Scrum Master, Product Owner, Dev Team), and how feedback/retrospectives are handled.

3.2 Requirements Gathering

3.2.1 User Stories

Write user stories in the format: "As a [role], I want to [action], so that [benefit]." Group by user role (Admin, Student, Faculty).

3.2.2 Functional Requirements

List system functions as numbered, testable statements (e.g., FR-01: The system shall allow a Student to submit a request).

3.2.3 Non-Functional Requirements

Cover performance, security, usability, and scalability requirements (e.g., NFR-01: The system shall hash all passwords before storage).

3.3 System Architecture Design

Present a labeled diagram of the overall architecture: client layer, server/API layer, database layer, and any third-party services. State the chosen pattern (e.g., MVC) explicitly and explain the data flow between layers.

3.4 UML Diagrams

3.4.1 Use Case Diagram

Show all actors (Admin, Student, Faculty, etc.) and their interactions with the system.

3.4.2 Activity Diagrams

Diagram the workflow for each major process (e.g., account registration, request submission and approval).

3.4.3 Class Diagram

Show system classes, attributes, methods, and relationships (association, aggregation, inheritance) consistent with your chosen tech stack.

3.4.4 Entity-Relationship Diagram (ERD)

Model the database schema: entities, attributes, primary/foreign keys, and cardinality of relationships.

3.5 UI/UX Design

3.5.1 High-Fidelity Prototype

Reference the Figma/Adobe XD prototype link and embed key screen mockups here, mapped to the user stories they satisfy.

3.5.2 Design Rationale

Briefly justify layout, color, and navigation choices in terms of accessibility and target-user needs.

3.6 Alpha Build

Document the coded, navigable frontend (HTML/CSS/JS or framework) delivered at Midterms, with a note on which screens are complete and which are still static/unwired to the backend.

CHAPTER 4 SYSTEM IMPLEMENTATION AND TESTING

4.1 Development Environment and Tech Stack

List final frontend, backend, database, and hosting technologies used, with version numbers, and justify any changes from the Midterm proposal.

4.2 Implementation Summary

Summarize how backend logic, database, and frontend were integrated. Reference the GitHub/GitLab repository link and describe the branching strategy and commit conventions used.

4.3 Software Testing

4.3.1 Unit Testing

Describe what was tested at the function/module level, the tools used, and pass/fail summary.

4.3.2 Integration Testing

Describe tests verifying that modules (frontend–backend–database) work together correctly.

4.3.3 User Acceptance Testing (UAT)

Describe the UAT process: number of testers, roles represented, tasks performed, and satisfaction/success metrics.

4.3.4 Bug Log

Insert a table logging bugs found: ID, description, severity, module, status, and resolution.

Bug ID	Description / Severity / Module / Status
BUG-001	[Description] — [Low/Med/High] — [Module] — [Resolved/Open]
BUG-002	[Description] — [Low/Med/High] — [Module] — [Resolved/Open]

4.4 Deployment and Publishing

State the live URL (or compiled executable/APK), hosting platform (Vercel, Heroku, AWS, Render, etc.), and steps taken to configure the production environment, including environment variables and CI/CD if used.

4.5 Results and Discussion

Present testing results (pass rates, performance metrics, UAT scores) with tables/charts, and discuss what they mean for the system's readiness.

CHAPTER 5 SUMMARY

5.1 Summary of Findings

Recap the problem, the solution built, and the key testing/deployment outcomes in a few paragraphs.

5.2 Conclusion

State whether the objectives in Chapter 1 (Section 1.3) were met, referencing them explicitly one by one.

5.3 Recommendations

Recommend future enhancements, scalability improvements, or features deferred due to time constraints — useful talking points for the Final Defense Q&A.

References

List all sources cited in Chapter 2 in the citation style required by our program (IEEE) in order of appearance.

Appendices

Appendix A — Source Code Repository

GitHub/GitLab repository link and instructions to run the project locally.

Appendix B — User Manual

Step-by-step guide, with screenshots, for each user role (Admin, Student, Faculty).

Appendix C — UAT Forms and Raw Results

Scanned or exported UAT questionnaires and raw scoring data.

Appendix D — Consultation / Adviser's Log

Dated record of adviser meetings across all three terms.

Appendix E — Curriculum Vitae

One-page CV for each team member.

Prelim Rubric (Project Proposal)

Criteria	Excellent (90-100%)	Satisfactory (75-89%)	Needs Improvement (<75%)
Problem Identification 50%	Solves a complex, highly relevant engineering/community problem.	Solves a standard problem; relevance is adequate but common.	Vague problem statement; lacks real-world application.
Scope & Requirements 30%	Requirements are clearly defined, measurable, and highly realistic.	Requirements are mostly clear but lack some technical specificity.	Scope is too broad or too narrow; poorly defined requirements.
Pitch Presentation 20%	Exceptional communication; confidently answered all technical questions.	Good presentation; struggled slightly with complex panel questions.	Unprepared; could not justify the project's technical feasibility.

Midterm Rubric (Documentation & Prototype)

Criteria	Excellent (90-100%)	Satisfactory (75-89%)	Needs Improvement (<75%)
System Architecture (UML) 50%	Diagrams are logically flawless, following strict industry standards.	Diagrams are mostly correct with minor logical inconsistencies.	Diagrams are incomplete, incorrect, or missing critical relationships.
UI/UX Prototype 30%	Interface is highly intuitive, modern, and perfectly maps to user stories.	Interface is functional and clean, but lacks some polish or intuitive flow.	Interface is clunky, hard to navigate, or visually unappealing.
Alpha Build Execution 20%	Frontend code is clean, responsive, and closely matches the prototype.	Frontend is functional but lacks responsiveness or has minor bugs.	UI code is broken, incomplete, or doesn't match the design.

Final Rubric (Development & Publishing)

Criteria	Excellent (90-100%)	Satisfactory (75-89%)	Needs Improvement (<75%)
System Functionality 30%	100% of core features work flawlessly. CRUD operations are robust and secure.	Most core features work; minor non-breaking bugs present.	Critical features are broken or missing; frequent system crashes.
Code Quality & Git Use 25%	Code is modular, well-commented. Git history shows consistent, clear team collaboration.	Code is decent but lacks comments. Git history is inconsistent.	Spaghetti code; no proper version control used.
Deployment & Publishing 25%	App is successfully deployed live, runs fast, and handles errors gracefully.	App is deployed but experiences latency or environment configuration errors.	App runs only locally; failed to deploy or publish correctly.
Final Defense & Q&A 20%	Flawless live demo. Team demonstrated deep mastery of their codebase and architecture.	Good demo. Team understood the code but struggled with advanced architectural questions.	Failed demo. Team could not explain how their own code works.